Assignment 2: Knowledge Representation and Reasoning

FIT5047: Fundamentals of AI — Semester 1, 2026

Juan Pulido
Student ID: 34169202
Faculty of Information Technology, Monash University
Email: jpul0007@student.monash.edu

Question 1: Propositional Logic — Advanced Validity and CNF Transformation

Given the formulas:

- $\phi_1 = ((A \vee (B \to C)) \wedge (\neg C \vee D)) \to ((A \to D) \vee (C \leftrightarrow E))$
- $\phi_2 = (((\neg A \wedge B) \vee (C \to D)) \wedge (\neg E \vee A)) \to ((\neg D \vee F) \leftrightarrow (A \wedge B))$
- $\phi_3 = ((A \leftrightarrow B) \vee (C \to D)) \wedge ((\neg A \vee \neg D) \to (F \wedge \neg G))$

Tasks:

- **(a)** For each formula: Determine whether the clause is valid, satisfiable but not valid, or unsatisfiable. Justify using truth assignments or semantic arguments.
- **(b)** Convert each formula into CNF. Identify Horn clauses where applicable.
- **(c)** For ϕ_2, assume F is always false:
  - (i) What is the effect on satisfiability?
  - (ii) Can you infer anything about A or B? Justify.

Part (a): Validity and Satisfiability

> **Lecture Recap:**
>
> Before we begin, let us establish the logical definitions and rules we will use throughout the solution:
>
> - **Valid:** A formula is true in *all* possible models (truth assignments).
> - **Unsatisfiable:** A formula is false in *all* possible models.
> - **Satisfiable:** A formula is true in *at least one* model.
> - **Satisfiable but not valid:** A formula is true in some models, but false in others.
> - **Horn Clause:** A disjunction of literals (an OR-statement) that contains **at most one positive literal** (e.g., $\neg A \vee \neg B \vee C$).

Strategy to prove validity and satisfiability

- To prove a formula is **satisfiable**, we only need to find one **model** that makes it $True$.
- To prove a formula is **not valid**, we must find a counterexample: a model that makes it $False$.
- Since all three formulas are structured as **implications** ($Antecedent \Rightarrow Consequent$), the only way to make the entire formula $False$ is to make the $Antecent = True$ and the $Consequent = False$.

1. Formula ϕ_1

$\phi_1 = ((A \vee (B \to C)) \wedge (\neg C \vee D)) \to ((A \to D) \vee (C \leftrightarrow E))$

Let $\alpha = ((A \vee (B \to C)) \wedge (\neg C \vee D))$ **the antecedent**
Let $\beta = ((A \to D) \vee (C \leftrightarrow E))$ **the consequent**
So $\phi_1 = \alpha \to \beta$

- **Satisfiability Check:**
  We can easily satisfy the implication by making the $\beta = True$.
  - Let $A = False$ and $D = True$.
  - In the β term $(A \to D)$ becomes $(False \to True) \equiv True$.
  - Since the β is $True$, the entire implication ϕ_1 evaluates to $True$.
  - Thus, ϕ_1 is **satisfiable**.
- **Validity Check:**
  We attempt to find a model where ϕ_1 is $False$ ($\alpha = True$, $\beta = False$).
  - For β to be $False$, both parts of the OR must be $False$:
    - $(A \to D)$ must be $False$ (meaning $A = True, D = False$)
    - $(C \leftrightarrow E)$ must be $False$ (meaning $C \neq E$)
  - For α to be $True$, both parts of the AND must be $True$.
    - Given $D = False$, the second term $(\neg C \vee False)$ means $\neg C$ must be $True$, so $C = False$.
  - Since $C \neq E$ from β and $C = False$ from α, then $E = True$.
  - Looking at the first term of the α: $(A \vee (B \to C))$.
    - Since we know $A = True$, this term is automatically $True$.
  - **Counter-Model:** $\{A = True, B = True, C = False, D = False, E = True\}$.
    - In this model, α is $True$ and β is $False$, yielding $True \to False \equiv False$.

🎯 **Conclusion for ϕ_1:** The formula is **Satisfiable but Not Valid**.

2. Formula ϕ_2

$\phi_2 = (((\neg A \wedge B) \vee (C \to D)) \wedge (\neg E \vee A)) \to ((\neg D \vee F) \leftrightarrow (A \wedge B))$

Let $\alpha = (((\neg A \wedge B) \vee (C \to D)) \wedge (\neg E \vee A))$ **the antecedent**
Let $\beta = ((\neg D \vee F) \leftrightarrow (A \wedge B))$ **the consequent**
So $\phi_2 = \alpha \to \beta$

- **Satisfiability Check:**
  We can satisfy ϕ_2 by making the $\alpha = False$.
  - Let $A = False$ and $E = True$.
  - The term $(\neg E \vee A)$ becomes $(False \vee False) \equiv False$.
  - Since the α is $False$, the main implication ϕ_2 evaluates to $True$.
  - Thus, ϕ_2 is **satisfiable**.
- **Validity Check:**
  We attempt to find a counterexample where α is $True$ and β is $False$.
  - Let $D = False$ and $F = False$.
    - The β becomes $(True \vee False) \leftrightarrow (A \wedge B) \equiv True \leftrightarrow (A \wedge B)$.
    - For β to be $False$, $(A \wedge B)$ must be $False$.
    - Let $A = False$, which satisfies $(A \wedge B) = False$.
  - Let $E = False$.
    - The term in α $(\neg E \vee A)$ becomes $(True \vee False) \equiv True$.
  - Now evaluate the remaining part of α: $((\neg A \wedge B) \vee (C \to D))$.
    - Since $A = False$, $\neg A = True$.
    - Let $B = True$, so $(\neg A \wedge B) \equiv True$.
    - Let $C = False$, so $(C \to False) \equiv True$.
    - The α is $True$.
  - **Counter-Model:** $\{A = False, B = True, C = False, D = False, E = False, F = False\}$.
    - In this model, α is $True$ and β is $False$, yielding $True \to False \equiv False$.

🎯 **Conclusion for ϕ_2:** The formula is **Satisfiable but Not Valid**.

3. Formula ϕ_3

$\phi_3 = ((A \leftrightarrow B) \vee (C \to D)) \wedge ((\neg A \vee \neg D) \to (F \wedge \neg G))$

Let $\alpha = ((A \leftrightarrow B) \vee (C \rightarrow D))$ **the left conjunct**
Let $\beta = ((\neg A \vee \neg D) \rightarrow (F \wedge \neg G))$ **the right conjunct**
So $\phi_3 = \alpha \wedge \beta$

- **Satisfiability Check:**
  - Let $A = True, B = True, D = True$.
  - **The** α**:** $(True \leftrightarrow True) \vee \cdots \equiv True$.
  - **The** β**:** The antecedent is $(\neg True \vee \neg True) \equiv False$.
    - An implication with a $False$ antecedent $(False \rightarrow \ldots) \equiv True$.
  - Since both α and β are $True$, ϕ_3 is $True$.
  - Thus, ϕ_3 is **satisfiable**.
- **Validity Check:**
  We can make ϕ_3 $False$ simply by making the $\alpha = False$.
  - Let $A = True, B = False$ (making $A \leftrightarrow B$ $False$).
  - Let $C = True, D = False$ (making $C \rightarrow D$ $False$).
  - The α is $False$, making the entire formula $\phi_3 = False$.

🎯 **Conclusion for** ϕ_3**:** The formula is **Satisfiable but Not Valid**.

Part (b): CNF Transformation and Horn Clauses

Lecture Recap

- **CNF Conversion:** Any logical sentence can be converted into **Conjunctive Normal Form (CNF)**, a conjunction of disjunctions (AND of ORs) by applying four theoretical steps:
  i. **Eliminate biconditionals** $\leftrightarrow$**:** $(A \leftrightarrow B) \equiv (A \rightarrow B) \wedge (B \rightarrow A)$
  ii. **Eliminate implications** $\rightarrow$**:** $(A \rightarrow B) \equiv (\neg A \vee B)$
  iii. **Move** $\neg$ **inwards:** Apply **De Morgan's laws** and **Double Negation**.
  iv. **Distribute** $\vee$ **over** $\wedge$**:** $A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C)$
- **Horn Clause:** A clause (disjunction of literals) containing **at most one positive literal**. These are critical for efficient inference in logical agents.

Transformation of ϕ_1

$\phi_1 = ((A \vee (B \rightarrow C)) \wedge (\neg C \vee D)) \rightarrow ((A \rightarrow D) \vee (C \leftrightarrow E))$

Let $\alpha = ((A \vee (B \rightarrow C)) \wedge (\neg C \vee D))$ **the antecedent**
Let $\beta = ((A \rightarrow D) \vee (C \leftrightarrow E))$ **the consequent**
So $\phi_1 = \alpha \rightarrow \beta$

- **Original**: $\alpha \rightarrow \beta$. Eliminate implication: $\neg\alpha \vee \beta$.
- **Process** α**:** $\alpha = ((A \vee \neg B \vee C) \wedge (\neg C \vee D))$.
  Apply De Morgan's to find $\neg\alpha$:
  $\neg\alpha = (\neg A \wedge B \wedge \neg C) \vee (C \wedge \neg D)$.
- **Process** β**:** $\beta = (\neg A \vee D) \vee (C \leftrightarrow E)$.
  Expand $\leftrightarrow$: $(\neg A \vee D) \vee ((\neg C \vee E) \wedge (\neg E \vee C))$.
  Distribute $\vee$: $\beta = (\neg A \vee D \vee \neg C \vee E) \wedge (\neg A \vee D \vee C \vee \neg E)$.
- **Combine and Simplify:** We distribute $\neg\alpha$ over the two β conjunctions:
  - First conjunct: $\neg\alpha = (\neg A \wedge B \wedge \neg C) \vee (C \wedge \neg D) \vee (\neg A \vee D \vee C \vee \neg E)$.
    Apply rule absorption: Notice $(C \wedge \neg D) \vee D \equiv C \vee D$.
    The expression simplifies to $\alpha = \neg A \vee C \vee D \vee \neg E$.
  - Second conjunct: $\beta = (\neg A \wedge B \wedge \neg C) \vee (C \wedge \neg D) \vee (\neg A \vee D \vee \neg C \vee E)$.
    Apply rule: $(C \wedge \neg D) \vee \neg C \equiv \neg C \vee \neg D$.
    This leaves us with $\beta = D \vee \neg D$, which is a tautology ($\equiv True$)
    So this entire clause drops out.
- **Final CNF for** ϕ_1**:** $(\neg A \vee C \vee D \vee \neg E)$.

- **Horn Clause Check:** It contains two positive literals (C, D), so it is **not** a Horn clause.

Transformation of ϕ_2

$\phi_2 = (((\neg A \wedge B) \vee (C \to D)) \wedge (\neg E \vee A)) \to ((\neg D \vee F) \leftrightarrow (A \wedge B))$

Let $\alpha = (((\neg A \wedge B) \vee (C \to D)) \wedge (\neg E \vee A))$ **the antecedent**
Let $\beta = ((\neg D \vee F) \leftrightarrow (A \wedge B))$ **the consequent**
So $\phi_2 = \alpha \to \beta$

Convert $\phi_2 = \alpha \to \beta \equiv \neg\alpha \vee \beta$

- We again apply $\neg\alpha \vee \beta$.
- **Process $\neg\alpha$:** $\neg(((\neg A \wedge B) \vee (\neg C \vee D)) \wedge (\neg E \vee A))$.
  Pushing negations yields: $\neg\alpha = (A \wedge C \wedge \neg D) \vee (\neg B \wedge C \wedge \neg D) \vee (E \wedge \neg A)$.
- **Process β:** $(D \vee F) \leftrightarrow (A \wedge B)$.
  Converting this generates 5 distinct sub-clauses:
  $(D \vee A) \wedge (D \vee B) \wedge (\neg F \vee A) \wedge (\neg F \vee B) \wedge (\neg A \vee \neg B \vee \neg D \vee F)$.
- **Combine and Simplify:** Distributing $\neg\alpha$ over the β yields 11 distinct clauses:
  - 1: $\neg A \vee \neg B \vee \neg D \vee F$ **(Horn clause)**
  - 2: $\neg B \vee A \vee D \vee E$
  - 3: $A \vee C \vee D \vee E$
  - 4: $B \vee C \vee D \vee E$
  - 5: $\neg A \vee B \vee C \vee D$
  - 6: $A \vee C \vee E \vee \neg F$
  - 7: $A \vee \neg D \vee E \vee \neg F$
  - 8: $B \vee C \vee E \vee \neg F$
  - 9: $\neg A \vee B \vee C \vee \neg F$
  - 10: $B \vee \neg D \vee E \vee \neg F$
  - 11: $\neg A \vee B \vee \neg D \vee \neg F$ **(Horn clause)**
- **Horn Clause Check:** Clauses 1 and 11 contain exactly one positive literal each (F and B, respectively), making them Horn clauses.
  - **Clause 1:** $(\neg A \vee \neg B \vee \neg D \vee F)$ (One positive literal: F)
  - **Clause 11:** $(\neg A \vee B \vee \neg D \vee \neg F)$ (One positive literal: B)

Transformation of ϕ_3

$\phi_3 = ((A \leftrightarrow B) \vee (C \to D)) \wedge ((\neg A \vee \neg D) \to (F \wedge \neg G))$

Let $\alpha = ((A \leftrightarrow B) \vee (C \to D))$ **the left conjunct**
Let $\beta = ((\neg A \vee \neg D) \to (F \wedge \neg G))$ **the right conjunct**
So $\phi_3 = \alpha \wedge \beta$

- Because ϕ_3 is an AND statement between two large expressions, we process them as α and β separately.
- **Process α:** $(A \leftrightarrow B) \vee (C \to D)$.
  - Eliminate equivalents: $((A \vee \neg B) \wedge (\neg A \vee B)) \vee (\neg C \vee D)$.
  - Distribute OR over AND: $(A \vee \neg B \vee \neg C \vee D) \wedge (\neg A \vee B \vee \neg C \vee D)$.
- **Process β:** $(\neg A \vee \neg D) \to (F \wedge \neg G)$.
  - Eliminate implication: $\neg(\neg A \vee \neg D) \vee (F \wedge \neg G)$.
  - Apply De Morgan's: $(A \wedge D) \vee (F \wedge \neg G)$.
  - Distribute OR over AND: $(A \vee F) \wedge (A \vee \neg G) \wedge (D \vee F) \wedge (D \vee \neg G)$.
- **Final CNF for ϕ_3 (combining all 6 clauses):**
  - 1: $A \vee \neg B \vee \neg C \vee D$
  - 2: $\neg A \vee B \vee \neg C \vee D$
  - 3: $A \vee F$
  - 4: $A \vee \neg G$ **(Horn clause)**
  - 5: $D \vee F$
  - 6: $D \vee \neg G$ **(Horn clause)**
- **Horn Clause Check:** Clauses 4 and 6 contain exactly one positive literal (A and D), so they are Horn clauses.

Part (c): Evaluating ϕ_2 when F is always False

> **Lecture Recap:**
>
> - **Vacuous Truth (Implications):** An implication ($P \to Q$) is evaluated as True unless the premise (P) is True and the conclusion (Q) is False. If the premise is False, the implication is **vacuously True**.
> - **Logical Inference (Entailment):** We say $\alpha \models \beta$ if β is True in every **interpretation** where α is True. If satisfying models exist with conflicting truth values for a variable, no definitive entailment exists.

Given $\phi_2 = (((\neg A \wedge B) \vee (C \to D)) \wedge (\neg E \vee A)) \to ((\neg D \vee F) \leftrightarrow (A \wedge B))$.

Let $\alpha = (((\neg A \wedge B) \vee (C \to D)) \wedge (\neg E \vee A))$ **the antecedent**
Let $\beta = ((\neg D \vee F) \leftrightarrow (A \wedge B))$ **the consequent**
So $\phi_2 = \alpha \to \beta$

Assume $F = False$.

(i) What is the effect on satisfiability?

- Substitute $F = False$ into the original implication ϕ_2.
- The $\beta = ((\neg D \vee F) \leftrightarrow (A \wedge B))$.
- With $F = False$,
- The β becomes $((\neg D \vee False) \leftrightarrow (A \wedge B))$, which simplifies directly to $(\neg D \leftrightarrow (A \wedge B))$.
- The modified formula ϕ_2' is: $\alpha \to (\neg D \leftrightarrow (A \wedge B))$.

> 🎯 **Conclusion:** The formula **remains satisfiable**. We can still trivially satisfy it by forcing the α to be $False$ (for example, setting $A = False$ and $E = True$), which makes the entire implication $True$ regardless of the β.

(ii) Can you infer anything about A or B? Justify

- Let us test different satisfying truth assignments (models) for our modified ϕ_2':
  - **Model 1 (A is False, B is True):**
    - Let $A = False, E = True$.
    - The α evaluates to $False$.
    - The implication ϕ_2' holds True.
    - Thus, A can be False.
  - **Model 2 (A is True, B is True):**
    - Let $A = True, B = True, C = False, D = False, E = True$.
    - The α becomes $True$.
    - The β becomes $\neg False \leftrightarrow (True \wedge True) \equiv True \leftrightarrow True \equiv True$.
    - The implication ϕ_2' becomes $True \to True$ holds True.
  - **Model 3 (A is True, B is False):**
    - Let $A = True, B = False, C = False, D = True, E = True$.
    - The α becomes $True$.
    - The β becomes $\neg True \leftrightarrow (True \wedge False) \equiv False \leftrightarrow False \equiv True$.
    - The implication ϕ_2' becomes $True \to True$ holds True.

> 🎯 **Conclusion: No, we cannot unconditionally infer anything about the truth values of A or B.** Because there exist perfectly valid, satisfying models where A and B can independently take either True or False values, the knowledge base is not strict enough to force a specific truth assignment for them.

Question 2 Problem Statement: Enhanced Forward/Backward Reasoning — Deceptive Rules and Sparse Inference

Q2.1: Logic Translation with Mixed Relevance

Convert the following 10 statements into propositional logic, using the symbol key:

- **S:** is Sick
- **V:** likely to Vomit
- **P:** looks Pale
- **T:** is Thirsty
- **H:** has High Temperature
- **NS:** has Not Slept well
- **NF:** does Not Feel well
- **F:** has Fever
- **W:** drinks Water
- **E:** is Exhausted
- **A:** has Allergy
- **M:** has Muscle Pain

Note: If the knowledge base does not have the symbol key, it means that you can ignore it.

- **R1.** If a person looks pale, is thirsty, and has not slept well, then they are sick.
- **R2.** If a person has a fever, then they have a high temperature.
- **R3.** If a person has a high temperature, then they are thirsty.
- **R4.** If a person is exhausted, then they have not slept well.
- **R5.** If a person has not slept well, then they look pale.
- **R6.** If a person looks pale, then they should drink water.
- **R7.** If a person has muscle pain, then they are not feeling well.
- **R8.** A person has an allergy if they sneeze in spring.
- **R9.** If a person is not thirsty and has slept well, then they either will not vomit or will drink water.
- **R10.** If a person is not exhausted and not thirsty, then they might either feel well or drink water.

Initial facts (observations):

- The person has a fever.
- The person is exhausted.

Task: Prove that the person is sick (S)

Q2.2: Forward Chaining
Refer to our lab format to complete the forward chaining proving process.

Q2.3: Backward Chaining
Refer to our lab format to complete the backward chaining proving process.

Q2.4 Discussion
(i) Does forward chaining stop early in this scenario? Why?
(ii) Could adding $\neg M$ or A improve inference? Why or why not?

Q2.1: Logic Translation with Mixed Relevance

> **Lecture Recap:**
>
> - **Propositional Logic:** We translate English sentences into formal logic using logical connectives:
> - Conjunction: AND / $\wedge$
> - Disjunction: OR / $\vee$
> - Implication: IF-THEN / $\Rightarrow$

- Negation: NOT / $\neg$
- **Implication:** A sentence structured as "**If A, then B**" or "B **if** A" is translated as $A \Rightarrow B$.
- **Handling Unrepresented Symbols:** According to the assignment instructions, if a statement contains elements not present in our designated symbol key, we must ignore it. This prevents the Knowledge Base (KB) from containing undefined or open variables that cannot be evaluated.

Propositional Logic Translation:

- **R1:** If a person looks pale (P), is thirsty (T), and has not slept well (NS), then they are sick (S).
  Formula: $P \wedge T \wedge NS \Rightarrow S$
- **R2:** If a person has a fever (F), then they have a high temperature (H).
  Formula: $F \Rightarrow H$
- **R3:** If a person has a high temperature (H), then they are thirsty (T).
  Formula: $H \Rightarrow T$
- **R4:** If a person is exhausted (E), then they have not slept well (NS).
  Formula: $E \Rightarrow NS$
- **R5:** If a person has not slept well (NS), then they look pale (P).
  Formula: $NS \Rightarrow P$
- **R6:** If a person looks pale (P), then they should drink water (W).
  Formula: $P \Rightarrow W$
- **R7:** If a person has muscle pain (M), then they are not feeling well (NF).
  Formula: $M \Rightarrow NF$
- **R8:** A person has an allergy (A) if they sneeze in spring.
  Formula: IGNORED.
  *Observation: "**Sneeze in spring**" is not in the provided symbol key. Per the assignment rules, because we lack a symbol for the premise, this rule must be discarded from our formal KB.*
- **R9:** If a person is not thirsty ($\neg T$) and has slept well (negation of NS: $\neg NS$), then they either will not vomit ($\neg V$) or will drink water (W).
  Formula: $\neg T \wedge \neg NS \Rightarrow \neg V \vee W$
- **R10:** If a person is not exhausted ($\neg E$) and not thirsty ($\neg T$), then they might either feel well (negation of NF: $\neg NF$) or drink water (W).
  Formula: $\neg E \wedge \neg T \Rightarrow \neg NF \vee W$

Initial Facts (Observations):

- The person has as fever (F)
- The person is exhausted (E)

Goal:

- Prove that the person is sick (S)

Q2.2: Forward Chaining

Lecture Recap:

- **Forward Chaining** is a **data-driven** algorithm. It starts with the known facts and applies **Modus Ponens** iteratively to trigger rules and generate new facts until the goal is reached or no more facts can be inferred.
- **Definite Clauses:** Forward chaining expects the knowledge base to consist of Definite Clauses (rules with exactly one positive literal in the conclusion). Rules $R9$ and $R10$ contain disjunctions (OR / $\vee$) in their conclusions, meaning they are not Horn clauses and cannot be processed by standard forward chaining. We will evaluate the counts for the valid definite clauses ($R1$ through $R7$).
- **Algorithm Mechanics:** We maintain a `Count` of how many premises are unmet for each rule. When a fact is popped from the `Queue`, we decrement the count for any rule requiring that fact. If a rule's count reaches 0, its conclusion is added to the `Queue`.

Execution Steps:

- **Initial state:**
  We initialize the `Count` for the premises of our definite clauses:

- $R1$ has 3 premises (P, T, NS).
- $R2, R3, R4, R5, R6, R7$ each has 1 premise.

- **Queue:**
  Populated with the initial facts: $[F, E]$.

Step	Queue	Count	Inferred	Action / Rule Triggered
0	F, E	R1:3, R2:1, R3:1, R4:1, R5:1, R6:1, R7:1		*Initial State*
1	E	R1:3, R2:0, R3:1, R4:1, R5:1, R6:1, R7:1	F	Pop F. R2 count reaches 0. Infer H. Add H to Queue.
2	E, H	R1:3, R2:0, R3:1, R4:0, R5:1, R6:1, R7:1	F, E	Pop E. R4 count reaches 0. Infer NS. Add NS to Queue.
3	H, NS	R1:3, R2:0, R3:0, R4:0, R5:1, R6:1, R7:1	F, E, H	Pop H. R3 count reaches 0. Infer T. Add T to Queue.
4	NS, T	R1:2, R2:0, R3:0, R4:0, R5:0, R6:1, R7:1	F, E, H, NS	Pop NS. R1 count drops to 2. R5 count reaches 0. Infer P. Add P to Queue.
5	T, P	R1:1, R2:0, R3:0, R4:0, R5:0, R6:1, R7:1	F, E, H, NS, T	Pop T. R1 count drops to 1.
6	P	R1:0, R2:0, R3:0, R4:0, R5:0, R6:0, R7:1	F, E, H, NS, T, P	Pop P. R1 count reaches 0. Infer S. R6 count reaches 0. Infer W. Add S, W to Queue.
7	S, W	R1:0, R2:0, R3:0, R4:0, R5:0, R6:0, R7:1	F, E, H, NS, T, P, S	Pop S. **Goal Reached!**

🎯 **Conclusion:** The algorithm successfully proves S (the person is Sick).

Q2.3: Backward Chaining

Lecture Recap:

- **Backward Chaining** is a **goal-driven** algorithm. It works backward from the query q (the Goal).
- **Mechanism:** To prove a goal, the algorithm checks if it is already a known fact. If not, it finds all rules that have the goal as their conclusion (the right-hand side of the $\Rightarrow$) and recursively attempts to prove every premise of those rules (the left-hand side) as new subgoals.
- **Efficiency:** It avoids loops by checking if a subgoal is already on the stack, and it avoids repeated work by checking if a subgoal has already been proven true in a previous branch.

Execution Steps:

1. **Current Goal:** S
   - Is S a known fact? No.
   - Find a rule concluding S: **R1** ($P \wedge T \wedge NS \Rightarrow S$).
   - To prove S, we must prove the subgoals: P, T, NS.
2. **Evaluate Subgoal 1:** P
   - Is P a known fact? No.
   - Find a rule concluding P: **R5** ($NS \Rightarrow P$).
   - New Subgoal to prove P: NS.
   - **Evaluate Subgoal 1.1:** NS
     - Is NS a known fact? No.
     - Find a rule concluding NS: **R4** ($E \Rightarrow NS$).
     - New Subgoal to prove NS: E.

    - **Evaluate Subgoal 1.1.1:** E
      - Is E a known fact? **Yes!**
    - *Result:* E is true $\rightarrow$ NS is true $\rightarrow$ P **is true**.
3. **Evaluate Subgoal 2:** T (Returning to R1's premises)
   - Is T a known fact? No.
   - Find a rule concluding T: **R3** ($H \Rightarrow T$).
   - New Subgoal to prove T: H.
   - **Evaluate Subgoal 2.1:** H
     - Is H a known fact? No.
     - Find a rule concluding H: **R2** ($F \Rightarrow H$).
     - New Subgoal to prove H: F.
     - **Evaluate Subgoal 2.1.1:** F
       - Is F a known fact? **Yes!**
     - *Result:* F is true $\rightarrow$ H is true $\rightarrow$ T **is true**.
4. **Evaluate Subgoal 3:** NS (Returning to R1's premises)
   - Is NS a known fact? No.
   - *Theory Check:* Has this subgoal already been proven? **Yes.** We proved NS in Step 2 while proving P.
   - *Result:* The algorithm immediately returns true without recalculating. NS **is true**.

🎯 **Conclusion:** All three premises (P, T, NS) of R1 evaluate to true. Therefore, the goal S is proven to be true.

Q2.4: Discussion

(i) Does forward chaining stop early in this scenario? Why?

Yes, forward chaining stops early.

- **Justification:**
  The forward chaining algorithm includes an explicit termination condition:
  - `if p == q then return true`
    - p is the fact currently popped from the queue
    - q is the goal.
  - In Step 6 of our trace, processing P inferred both S and W. Both were added to the queue.
  - In Step 7, S was popped from the queue. Because S matched our goal, the algorithm successfully stopped immediately.
  - It *stopped early* because it did not waste computational resources evaluating the remaining items in the queue (like popping W) or attempting to trigger further rules down the line.
  - It also naturally ignored irrelevant paths (like $R7$) because they were never triggered by the data.

(ii) Could adding $\neg M$ or A improve inference? Why or why not?

No, adding $\neg M$ or A would not improve inference.

- **Justification:**
  - **Relevance to Goal:** The goal S is already perfectly provable using only the minimal initial facts (F, E). Adding more facts to the KB cannot make the proof of S any more successful; it would only expand the state space and the queue size, mathematically *decreasing* algorithmic efficiency.
  - **Structural Limitations:** Standard Forward and Backward Chaining algorithms are designed to operate exclusively on Horn Knowledge Bases (Definite Clauses containing positive literals). Adding a negative literal ($\neg M$) cannot be utilized by basic chaining rules, and rule $R7$ requires a positive M to trigger.
  - **Disconnected Rules:** The symbol A relies on rule $R8$. As established in Q2.1, R8 is discarded because its premise ("**sneeze in spring**") is unrepresented. Consequently, A is entirely disconnected from the logic network mapping to S. Injecting it provides zero heuristic or logical value.

Question 3: First-Order Logic Multi-Clause Resolution Challenge

We have the following clauses:

1. $\exists z.R(z) \rightarrow \exists w.T(w)$
2. $\forall x \exists y.(\forall z.(P(x) \vee R(z)) \wedge \exists w.(T(w) \rightarrow Q(y)))$
3. $\forall w \exists y.(\forall x.(P(x) \rightarrow T(w)) \wedge \forall x.(Q(y) \rightarrow P(x)))$
4. $\forall x \exists z.(\neg R(z) \vee \neg Q(x))$
5. $\forall x.(U(x) \rightarrow (P(x) \vee \neg T(f(x))))$
6. $\exists x.(Q(x) \wedge \neg U(x))$
7. $\forall x \forall y.(\neg P(x) \vee Q(y) \vee S(x, y))$

Goal (G): $\forall x.P(x)$

Tasks:

- **(a)** Convert all into CNF. Show Skolemization and standardization.
- **(b)** Apply resolution to prove or refute G.
- **(c)** Identify (i) which clauses are redundant and (ii) which ones increase complexity the most. Provide justifications.
- **(d)** Suppose clause 6 is removed. Does this impact provability of G? Justify.

Part (a): Convert all into CNF. Show Skolemization and standardization.

> **Lecture Recap:**
> To convert any well-formed formula (wff) into Conjunctive Normal Form (CNF) clauses, we must strictly apply the following 9-step algorithm:
>
> 1. **Eliminate implication symbols:** Apply the equivalence $(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$ and $(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$.
> 2. **Reduce the scopes of negation symbols (¬):** Push negations inward using De Morgan's Laws $\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$ and $\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$, double negation $\neg(\neg\alpha) \equiv \alpha$, and quantifier equivalencies $\neg(\forall x)P(x) \equiv (\exists x)\neg P(x)$ and $\neg(\exists x)P(x) \equiv (\forall x)\neg P(x)$.
> 3. **Standardise variables:** Rename variables so that each quantifier binds a uniquely named variable.
> 4. **Eliminate existential quantifiers (∃) (Skolemise):** Replace existential variables with a **Skolem constant** if they do not depend on any universal quantifier (∀), or a **Skolem function** (e.g., $f(x)$) if they are inside the scope of a universal quantifier and thus depend on it.
> 5. **Move all universal quantifiers (∀) to the front.**
> 6. **Put the result in CNF:** Distribute OR (∨) over AND (∧) using the equivalency $(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$.
> 7. **Eliminate universal quantifiers (∀).**
> 8. **Eliminate conjunction symbols (∧):** Split the formula into separate, distinct clauses.
> 9. **Rename variables (Standardise variables apart):** Rename variables so no two final clauses share the exact same variable names.
>
> **Proof by Refutation:** To prove a goal using resolution, we must first negate the goal and add it to our knowledge base. If we can derive a contradiction (an empty clause, `NIL`), the original goal is proven.

Goal (G): $\forall x.P(x)$

- **Negate the Goal:** $\neg(\forall x.P(x))$
- **Step 2 (Reduce Negation):** Apply equivalence $\neg(\forall x)P(x) \equiv (\exists x)\neg P(x)$.
  Result: $\exists x.\neg P(x)$
- **Step 4 (Skolemise):** The variable x is not dependent on any universal quantifier. We replace x with a **Skolem constant** C.
  Result: $\neg P(C)$
- **Steps 5-9:**
  Clause G.1: $\neg P(C)$

Clause 1: $\exists z.R(z) \Rightarrow \exists w.T(w)$

- **Step 1 (Eliminate ⇒):** Apply $(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$.
  Result: $\neg(\exists z.R(z)) \vee \exists w.T(w)$

- **Step 2 (Reduce $\neg$):** Apply $\neg(\exists z)P(z) \equiv (\forall z)\neg P(z)$.
  Result: $(\forall z.\neg R(z)) \vee \exists w.T(w)$
- **Step 3 (Standardise):** Variables z and w are already unique.
- **Step 4 (Skolemise):** The existential quantifier $\exists w$ is strictly outside the scope of $\forall z$. It depends on nothing. We replace w with a **Skolem constant** A.
  Result: $(\forall z.\neg R(z)) \vee T(A)$
- **Step 5 (Move $\forall$ outward):**
  Result: $\forall z.(\neg R(z) \vee T(A))$
- **Step 6 & 7 (CNF & Drop $\forall$):**
  Result: $\neg R(z) \vee T(A)$
- **Step 9 (Standardise apart):** Rename z to z_1.
  Clause 1.1: $\neg R(z_1) \vee T(A)$

Clause 2: $\forall x \exists y.(\forall z.(P(x) \vee R(z)) \wedge \exists w.(T(w) \rightarrow Q(y)))$

- **Step 1 (Eliminate $\rightarrow$):** Apply $(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$ to the inner implication.
  Result: $\forall x \exists y.(\forall z.(P(x) \vee R(z)) \wedge \exists w.(\neg T(w) \vee Q(y)))$
- **Step 2 (Reduce $\neg$):** No negations to push inward.
- **Step 3 (Standardise):** Variables x, y, z, w are already uniquely bound by their respective quantifiers.
- **Step 4 (Skolemise):**
  - The existential variable y is strictly inside the scope of the universal variable x. We replace y with a **Skolem function** $f_1(x)$.
  - The existential variable w is also inside the scope of x, but crucially, it is *outside* the scope of $\forall z$. Therefore, w depends *only* on x, not z. We replace w with a **Skolem function** $f_2(x)$.
    Result: $\forall x.(\forall z.(P(x) \vee R(z)) \wedge (\neg T(f_2(x)) \vee Q(f_1(x))))$
- **Step 5 (Move $\forall$ outward):**
  Result: $\forall x \forall z.((P(x) \vee R(z)) \wedge (\neg T(f_2(x)) \vee Q(f_1(x))))$
- **Step 6 (CNF):** The formula is already a conjunction of disjunctions.
- **Step 7 (Drop $\forall$):**
  Result: $(P(x) \vee R(z)) \wedge (\neg T(f_2(x)) \vee Q(f_1(x)))$
- **Step 8 (Split into clauses):**
  Clause 2a: $P(x) \vee R(z)$
  Clause 2b: $\neg T(f_2(x)) \vee Q(f_1(x))$
- **Step 9 (Standardise apart):**
  Clause 2.1: $P(x_2) \vee R(z_2)$
  Clause 2.2: $\neg T(f_2(x_3)) \vee Q(f_1(x_3))$

Clause 3: $\forall w \exists y.(\forall x.(P(x) \rightarrow T(w)) \wedge \forall x.(Q(y) \rightarrow P(x)))$

- **Step 1 (Eliminate $\rightarrow$):**
  Result: $\forall w \exists y.(\forall x.(\neg P(x) \vee T(w)) \wedge \forall x.(\neg Q(y) \vee P(x)))$
- **Step 3 (Standardise):** The variable x is quantified twice in parallel scopes. To safely move the universal quantifiers outward in Step 5 without scope collision, we must rename the second x to a new variable, v.
  Result: $\forall w \exists y.(\forall x.(\neg P(x) \vee T(w)) \wedge \forall v.(\neg Q(y) \vee P(v)))$
- **Step 4 (Skolemise):** y depends on $\forall w$. Replace y with a **Skolem function** $f_3(w)$.
  Result: $\forall w.(\forall x.(\neg P(x) \vee T(w)) \wedge \forall v.(\neg Q(f_3(w)) \vee P(v)))$
- **Step 5 (Move $\forall$ outward):**
  Result: $\forall w \forall x \forall v.((\neg P(x) \vee T(w)) \wedge (\neg Q(f_3(w)) \vee P(v)))$
- **Steps 6, 7 & 8 (CNF, Drop $\forall$ & Split):**
  Clause 3a: $\neg P(x) \vee T(w)$
  Clause 3b: $\neg Q(f_3(w)) \vee P(v)$
- **Step 9 (Standardise apart):**
  Clause 3.1: $\neg P(x_4) \vee T(w_4)$
  Clause 3.2: $\neg Q(f_3(w_5)) \vee P(v_5)$

Clause 4: $\forall x \exists z.(\neg R(z) \vee \neg Q(x))$

- **Step 4 (Skolemise):** z depends on x. We use a **Skolem function** $f_4(x)$.
  Result: $\forall x.(\neg R(f_4(x)) \vee \neg Q(x))$
- **Steps 7-9 (Drop $\forall$, Split, Standardise apart):**
  Clause 4.1: $\neg R(f_4(x_6)) \vee \neg Q(x_6)$

Clause 5: $\forall x.(U(x) \rightarrow (P(x) \vee \neg T(f(x))))$

- **Step 1 (Eliminate $\rightarrow$):**
  Result: $\forall x.(\neg U(x) \vee P(x) \vee \neg T(f(x)))$
- **Steps 7-9 (Drop $\forall$, Split, Standardise apart):**
  Clause 5.1: $\neg U(x_7) \vee P(x_7) \vee \neg T(f(x_7))$

Clause 6: $\exists x.(Q(x) \wedge \neg U(x))$

- **Step 4 (Skolemise):** $\exists x$ is not inside any universal scope. Replace x with a **Skolem constant** B.
  Result: $Q(B) \wedge \neg U(B)$
- **Steps 8 & 9 (Split & Standardise):** No variables remain to Standardise.
  Clause 6.1: $Q(B)$
  Clause 6.2: $\neg U(B)$

Clause 7: $\forall x \forall y.(\neg P(x) \vee Q(y) \vee S(x, y))$

- **Steps 7-9 (Drop $\forall$, Split, Standardise apart):**
  Clause 7.1: $\neg P(x_8) \vee Q(y_8) \vee S(x_8, y_8)$

Part (b): Apply resolution to prove or refute G

> **Lecture Recap:**
>
> The **Resolution Inference Rule** combines two clauses by eliminating a pair of complementary literals (L and $\neg L$) using a Most General Unifier (MGU) substitution θ. The resolvent is formed by joining the remaining literals. We aim to derive an empty clause (NIL), meaning a contradiction is found.

We can achieve the proof rapidly using just four clauses from our knowledge base:

1. **Resolve Clause 6.1 with Clause 4.1:**
   - **Parent Clauses:** $Q(B)$ and $\neg R(f_4(x_6)) \vee \neg Q(x_6)$
   - **Complementary Literals:** Q and $\neg Q$
   - **Unification (MGU):** Substitute x_6 with constant B. $\theta_1 = \{x_6/B\}$.
   - **Resolvent 8:** $\neg R(f_4(B))$
2. **Resolve Resolvent 8 with Clause 2.1:**
   - **Parent Clauses:** $\neg R(f_4(B))$ and $P(x_2) \vee R(z_2)$
   - **Complementary Literals:** $\neg R$ and R
   - **Unification (MGU):** Substitute variable z_2 with the function $f_4(B)$. $\theta_2 = \{z_2/f_4(B)\}$.
   - **Resolvent 9:** $P(x_2)$
3. **Resolve Resolvent 9 with Negated Goal (G.1):**
   - **Parent Clauses:** $P(x_2)$ and $\neg P(C)$
   - **Complementary Literals:** P and $\neg P$
   - **Unification (MGU):** Substitute x_2 with constant C. $\theta_3 = \{x_2/C\}$.
   - **Resolvent 10:** `NIL`

> 🎯 **Conclusion:** We successfully derived a contradiction (NIL) from the negated goal. Therefore, the original goal $\forall x.P(x)$ is **proven**.

Part (c): Clause Analysis

(i) Which clauses are redundant?

Clauses **1, 3, 5, 6.2, 7, and 2.2** are completely redundant.

Justification:
In logic, a literal is "pure" if it appears with only one polarity (either always positive or always negative) across all clauses in the knowledge base. Resolution requires unifying opposite polarities to eliminate them.
* Predicate $S(x, y)$ only appears *positively* (in Clause 7.1).
* Predicate U only appears *negatively* ($\neg U$ in Clauses 5.1 and 6.2).
Because these literals lack opposing polarities, any clause containing them can never be resolved away to generate an empty clause. Therefore, Clauses **5.1, 6.2, and 7.1** are strictly mathematically useless for a refutation proof. Furthermore, since we generated an elegant proof only utilizing Clauses 2.1, 4.1, 6.1, and G.1, Clauses **1, 3, and 2.2** are functionally redundant for this specific goal.

(ii) Which ones increase complexity the most?

Clauses **2, 3, 4, and 5** increase computational complexity the most.

Justification:
These clauses introduce **Skolem functions** ($f_1(x)$, $f_2(x)$, $f_3(w)$, $f_4(x)$, and $f(x)$). In automated theorem proving, functions exponentially inflate the *Herbrand Universe* (the set of all possible term substitutions, e.g., substituting x with $f(A)$, $f(f(A))$, etc.). This creates a massive, potentially infinite branching factor during the Unification algorithm, drastically slowing down the resolution search space. Clause 2 specifically increases complexity the most because its nested quantifiers force the formula to introduce two different Skolem functions *and* branch into two separate CNF clauses via distribution.

Part (d): Suppose clause 6 is removed. Does this impact provability of G?

Yes, removing Clause 6 makes G strictly unprovable.

Justification:
Resolution-refutation requires grounding out logical chains with definitive, unconditional facts. Clause 6 provides the *only* standalone positive fact in the entire knowledge base: $Q(B)$ (Clause 6.1).

- If we remove Clause 6, we strip the knowledge base of its only unconditioned positive Q. To refute the negated goal $\neg P(C)$, we must resolve it against a clause that provides a positive P, such as Clause 2.1 ($P(x_2) \vee R(z_2)$). However, this resolution leaves a residual "debt" of positive literals (like $R(z_2)$) that must be canceled.
- If you trace the dependencies, canceling $R(z_2)$ using Clause 4.1 ($\neg R \vee \neg Q$) requires a positive Q to cancel the resulting $\neg Q$. Without Clause 6.1, the only other sources of a positive Q are Clause 7.1 (which is a pure literal trap due to S) and Clause 2.2 ($\neg T \vee Q$). Using Clause 2.2 introduces $\neg T$, which demands a positive T, introducing new negative literals ($\neg R$ or $\neg P$), throwing the inference engine into an infinite loop.
- Furthermore, attempting to resolve the remaining $\neg Q(f_3(w_5))$ (from 3.2) against $Q(f_1(x_3))$ (from 2.2) requires unifying distinct Skolem functions (f_1 and f_3), which mathematically fails because different functions cannot be unified. Without Clause 6.1, the algorithm is permanently starved of the positive facts required to reach `NIL`.

Generative AI Acknowledgements

In accordance with the Monash University guidelines for the responsible use of Generative AI, the author acknowledges the use of AI tools during the preparation and documentation of this assignment.

- **NotebookLM** was utilised as a study assistant to review lecture materials, enhance the understanding of definitions and exercises provided in the lectures, and provide structural guidance for outlining the solutions.
- **Google Gemini** was employed in several capacities throughout the documentation process. First, it was used to scan handwritten drafts of the solutions to obtain suggestions on how to document the problem-solving steps more clearly. Second, Gemini assisted in formatting the final assignment solutions into Markdown code, leveraging LaTeX mathematical extensions for VS Code. Finally, Gemini was used to improve English grammar and phrasing.

All AI-generated suggestions, formatting, and text were rigorously reviewed, modified, and validated by the author to ensure mathematical correctness. The final submission represents a sincere demonstration of my own human effort, understanding, and subject knowledge.

Appendix: AI Prompts Used
As per the assignment requirements, the following prompts were used during the generation of this assessment:

- **NotebookLM Prompt (Concept Review and Outlining):** "Based on the FIT5047 lecture slides, can you explain the 9-step algorithm for converting First-Order Logic into Conjunctive Normal Form (CNF)? Please specifically clarify how 'Skolemization' and 'Moving universal quantifiers outward' work, and provide an outline of how I should structure my written solution for a resolution-refutation proof."
- **Gemini Prompt 1 (Handwritten Draft Review):** "I have uploaded an image of my handwritten step-by-step solution for the Forward and Backward Chaining problem. Can you review my logic and suggest how I can document this step-by-step process more clearly and professionally?"
- **Gemini Prompt 2 (Markdown and LaTeX Formatting):** "Please convert the following plain-text logical steps into clean Markdown format, leveraging LaTeX math extensions for VS Code. Ensure that all First-Order Logic symbols like universal quantifiers (\forall), existential quantifiers (\exists), implications (\Rightarrow), and logical connectives are correctly formatted in math blocks."
- **Gemini Prompt 3 (Grammar and Phrasing):** "Please proofread the following paragraph discussing why forward chaining stops early and why adding the literal $\neg M$ does not improve inference. Correct any English grammar mistakes and elevate the phrasing to sound more academic and technically precise."